

Scalp Terminal

Live Order Flow & Fill-Resolution Design

How every strategy places, protects, and records a live trade

Covering BB_V1 · BB_V2 · SCALP_V1 · SCALP_V2 · HA_V1

NSE Options (NIFTY / BANKNIFTY) · Zerodha Kite Connect

Technical & Operational Reference

Written for both newcomers and engineers

Contents

1. How to Read This Document

This document explains, in detail, how the Scalp Terminal application turns a trading signal into a real order at the broker — and, just as importantly, how it keeps that order honest: recording the price you actually got, protecting the position, and cleaning up safely when something goes wrong.

If you are new to the system: start with Section 2 (the plain-English overview) and Section 3 (the concepts and vocabulary). Each strategy section then opens with a short “in plain words” summary before the technical detail. You can stop reading at the end of each plain-words summary and still understand what the system does.

If you are an engineer: Section 3 defines the shared building blocks; Sections 5–9 give the per-strategy order lifecycle, the exact files and functions involved, the edge cases handled, and the audit-log breadcrumbs to look for. Section 10 is the consolidated edge-case matrix and Section 11 lists every file touched.

A note on scope

This document describes the live order-execution path: what happens from the moment a strategy decides to trade to the moment the position is recorded and protected. It does not cover signal generation logic (when/why a strategy decides to trade), backtesting, or the user interface, except where they touch order flow.

2. The Big Picture (Plain English)

Imagine you press “buy” on a stock app. Three things have to happen, in the right order, for that to be safe:

1. **Place the order.** Tell the broker you want to buy (or sell) a specific option at a specific price.
2. **Find out what actually happened.** Orders do not always fill instantly or at the exact price you asked for. You need to wait and confirm the real “fill price.”
3. **Protect the position.** Put a safety net under the trade — an automatic exit if the price moves against you (a stop-loss) or in your favour to your target (a take-profit).

The bug this system was carrying is subtle but expensive: it was recording the price it asked for instead of the price it actually got. Those two are usually close, but not identical — and on a fast-moving options contract they can differ enough to make your profit-and-loss numbers wrong, trade after trade.

The core fix in one sentence

Every strategy now waits for the broker to confirm the real fill price and records that, instead of trusting the price it originally asked for — while making sure the safety net (the protective order) is never placed on a trade that does not actually exist.

Because the five strategies run very differently — some place one order, one places three at once, some use a broker-side safety net and one manages exits itself — the same idea is implemented in three different shapes. The rest of this document explains each one.

The five strategies at a glance

Strategy	What it trades	In plain words
BB_V1	BANKNIFTY monthly options (buying)	The main money-maker. Buys options when Bollinger Band / pivot conditions line up. Uses a broker safety net (GTT).
BB_V2	BANKNIFTY (buying)	A tuned variant of BB_V1 that shares all the same order machinery.
SCALP_V1	NIFTY/BANKNIFTY options (selling)	Fast option-selling scalper. One position at a time. Broker safety net inverted for short positions.
SCALP_V2	NIFTY options (selling)	Like SCALP_V1 but fans one signal into three nearby strikes and exits them all together.
HA_V1	NIFTY weekly options (buying)	Heikin-Ashi strategy. Manages its own exits tick-by-tick instead of using a broker safety net.

3. Core Concepts & Vocabulary

These terms appear throughout the document. Skim them once; refer back as needed.

3.1 Trading & broker terms

Term	What it means
LTP (Last Traded Price)	The most recent price the option traded at. The app's reference for “where is the market right now.”
Limit order	An order to buy/sell only at a stated price or better. The app must use limit orders (not market orders) for regulatory protection (SEBI). A BUY limit is set slightly above LTP, a SELL limit slightly below, to maximise the chance of filling.
Fill / fill price / average price	The price at which the order actually executed at the exchange. Reported by the broker as <code>average_price</code> . This is the truth; the limit price is only a request.
Order status	The broker's verdict on an order: COMPLETE (filled), OPEN / TRIGGER PENDING (still working), or one of the dead states REJECTED / CANCELLED / LAPSED.
GTT (Good-Till-Triggered)	A resting order held at the broker that fires only when price crosses a trigger. Used as a hands-off safety net. An OCO GTT holds two triggers at once — a stop-loss and a take-profit — and whichever fires first cancels the other.
SL / TP	Stop-Loss (exit to cap a loss) and Take-Profit (exit to bank a gain).
LONG vs SHORT	LONG = you bought, you profit if price rises. SHORT = you sold (an option you don't own), you profit if price falls. P&L formulas differ: LONG = (exit – entry) × qty; SHORT = (entry – exit) × qty.
Partial fill	Only part of your order quantity executed (e.g. 30 of 60). The rest is still pending or cancelled.

3.2 System terms

Term	What it means
Tick	A single price update streamed from the broker over a WebSocket. Strategies react to ticks.
Tick thread	The single thread that receives and dispatches every incoming price tick. If this thread is blocked, the app stops reacting to prices — so nothing slow should ever run on it.
LTPStore	An in-memory cache of the latest tick price per symbol. Fast, but can be briefly stale, so it is never the sole source of an exit price.
Relay	An HTTP order-forwarding server on a fixed IP (Oracle Cloud / DigitalOcean) used for SEBI static-IP compliance. Orders are sent through the relay; reads (positions, order status) are always direct.
Audit log	The system's running diary. Every meaningful step writes a tagged line (e.g. [SHORT][FILL_CONFIRMED]). This document tells you which tags to look for.
Background thread	Work done off to the side so the main flow isn't blocked. The fill-confirmation work runs here.
Reconcile	On startup (and periodically), the app compares its own records against the broker's actual positions and corrects any mismatch.

3.3 The three execution modes

Every strategy can run in one of three modes, read live from configuration so it can change without a restart (with one safety exception):

- **LIVE** — real orders at the broker with real money.
- **PAPER** — simulated orders recorded in a database; no broker contact. Used for testing logic safely.
- **OFF** — the strategy still watches the market, builds candles and indicators, but takes no new entries. Existing positions are still managed to a clean exit.

Safety rule on mode changes

You can downgrade **LIVE** → **PAPER** mid-session, but you cannot upgrade **PAPER** → **LIVE** without a restart (the live broker state has to be built up cleanly first). **OFF** always takes effect immediately — turning a strategy off must never be blocked.

4. The Problem and the Three Fix Shapes

4.1 The original bug

When a strategy placed an order, it recorded the price it asked for (the limit price, or in some paths a cached LTP) as the trade's entry price — because the code either did not wait for the real fill, or could not tell “not filled yet” apart from “filled.” Consequences:

- **Wrong P&L.** Recorded entry $\neq$ real fill, so every trade's profit/loss was slightly off — worse for SHORT strategies where the entry feeds the (entry – exit) formula directly.
- **Phantom trades.** A rejected order could still leave the app thinking it held a position — and, worse, place a protective order on a position that never existed.
- **Frozen monitoring (HA).** One strategy waited up to 120 seconds for a fill on the tick thread, freezing price monitoring for its entire universe of contracts.

4.2 The shared building block

All five strategies now lean on one new, strategy-agnostic capability added to the executor: `get_order_fill(order_id)`. In a single call it reads the broker order book and returns the order's status, its average fill price, and how much has filled so far. This lets every strategy tell apart three situations that the old code blurred together: **filled** (COMPLETE), **dead** (REJECTED / CANCELLED / LAPSED), and **still working** (anything else, including “not in the order book yet”).

Why “not found yet” means “wait,” never “rejected”

The broker's order book can lag by seconds (occasionally minutes). If the app treated a momentarily-missing order as rejected, it might re-trade or wrongly abort. So a missing order is always treated as still-pending, never as dead. Only an explicit REJECTED / CANCELLED / LAPSED status is treated as dead.

4.3 Why three different shapes?

The strategies differ in two ways that force different designs: whether they use a broker-side GTT, and what kind of thread their entry runs on. The table summarises the approach taken for each.

Shape	Used by	Approach	Why
Block-until-fill	BB_V1, BB_V2	Entry waits (up to 120s) for COMPLETE, then places the GTT and records the true fill. Aborts cleanly on a dead order.	BB makes infrequent entries on monthly options; briefly blocking that one side's signal processing is acceptable and simplest.
Fill-first, then GTT	SCALP_V1, SCALP_V2	Place sell, record the limit price provisionally, confirm the fill in the background, then place the GTT only after a confirmed fill. Cancel the order if unfilled after 50s.	Selling needs the safety net to never fire on an order that didn't fill (which would open an unintended position). A scalp signal is only valid for

Shape	Used by	Approach	Why
			~1 candle, hence the 50s cancel.
Background-confirm, no GTT	HA_V1	Place buy, record provisionally, start monitoring immediately, confirm the fill in the background. HA has no GTT — it manages TP/SL itself.	HA's exits run on every tick; the entry must never block the tick thread, or TP monitoring for the whole universe would freeze.

5. BB_V1 — Bollinger Band Breakout (Option Buying)

In plain words

BB_V1 is the flagship strategy and the only one that has been running live daily. It buys BANKNIFTY monthly-expiry options when its signal conditions line up. It can split a position into two halves (a first target and a runner). For protection it places a broker-side GTT — a resting safety net that will sell automatically at either the stop-loss or the take-profit.

The fix: when BB buys, it now waits for the broker to confirm the real fill price before it sets up the safety net and records the trade. If the buy is rejected, it walks away cleanly instead of protecting a position that was never opened.

5.1 The live entry sequence

1. **Signal arrives.** The engine hands a confirmed ENTER_CE / ENTER_PE signal to the trade manager.
2. **Pre-checks.** Risk guard (daily max-loss), session window, and “already in a trade on this side?” are checked. Any failure aborts before any order is placed.
3. **Select the option.** From the futures price, the selector picks the strike to trade and a premium estimate.
4. **Place ONE buy order** for the total quantity (both legs combined), through the relay.
5. **Wait for the real fill (the fix).** The entry polls the broker order book until the order is COMPLETE, up to 120 seconds, then uses the true average fill price.
6. **Compute SL / TP** from the confirmed fill, place one or two GTTs (one per leg), and insert the database row(s).
7. **Register and notify.** Record the trade in memory and on disk, send the Telegram entry alert.

What the fill-resolution step decides

Broker says...	The strategy...	Audit-log tag
COMPLETE	Records the true average fill price, then places GTT(s) and DB row(s).	[BB][LIVE] [FILL_CONFIRMED]
REJECTED / CANCELLED / LAPSED	Aborts: no GTT, no DB row, returns failure, sends a warning alert. No phantom trade.	[BB][LIVE] [ENTRY_ORDER_DEAD]
Still working after 120s	Falls back to a best-effort price (REST LTP, then the selector premium), places the GTT anyway so a possibly-live position is protected, and logs loudly for reconcile to fix.	[BB][LIVE] [ENTRY_FILL_TIMEOUT]

Why timeout still places a GTT

If the order never confirms within 120s, it might still fill afterwards — leaving a real position. Rather than abandon it unprotected, BB protects it with a best-effort price and flags it; the next startup reconcile corrects the recorded price against the broker. Protecting a possibly-real position is safer than ignoring it.

5.2 Partial profit booking (two legs)

When multiple targets are enabled, BB_V1 buys the full quantity in one order, then splits it into two database rows (slots like PE_L1 and PE_L2) and places two GTTs: leg 1 exits at the first target, leg 2 (the runner) at a higher target. Both share the same stop-loss. If leg 1's target is hit, leg 2 keeps running; the state manager tracks both independently and reconciles partial exits on startup.

5.3 Exit flow

Exits (on a SuperTrend reversal signal, or at end-of-day square-off) cancel any live GTTs, place a market sell for the remaining quantity, clear in-memory state immediately, and confirm the sell fill in a background thread before writing the close to the database. The exit uses a fresh REST LTP as the price reference rather than the potentially-stale cache.

Files involved

File	Role in this flow
<code>bb_trade_manager.py</code>	Entry/exit lifecycle. Contains the reworked <code>_resolve_fill()</code> (status-based poll) and the <code>_enter</code> / <code>_exit</code> logic.
<code>bb_trade_state_manager.py</code>	Two-leg state, persistence to disk, and startup reconciliation against broker positions.
<code>zerodha_executor.py</code>	<code>place_buy</code> , <code>place_gtt_oco</code> , <code>place_market_sell</code> , and the new <code>get_order_fill</code> .
<code>trades_repo.py</code>	Database insert / update / close and direction-aware realized-P&L query.
<code>bb_tick_engine.py</code>	Constructs the BB_V1 trade manager and feeds it signals.

6. BB_V2 — Tuned Bollinger Variant

In plain words

BB_V2 is a tuned version of BB_V1 (a tighter SuperTrend multiplier and pivot-crossover entries). For the purposes of order flow there is nothing new to learn: BB_V2 sends its signals through the exact same trade manager as BB_V1.

6.1 Why BB_V2 needed no separate fix

BB_V2's differences live entirely in its **signal engine** (how it decides to trade). When it has decided, it hands the signal to the same shared BBTradeManager that BB_V1 uses. That manager is where the fill-resolution fix lives, so BB_V2 inherits it automatically. A repository check confirmed BB_V2 has no entry, fill, or order-placement code of its own — only its own signal/indicator logic and its own database columns (suffixed _v2).

Verification performed

Two call sites construct the shared manager — one in `bb_tick_engine.py` (BB_V1), one in `bb_tick_engine_v2.py` (BB_V2). A search for any separate BB_V2 fill/entry path returned nothing. Conclusion: the fix applies to both with no duplicate code.

Files involved

File	Role in this flow
<code>bb_tick_engine_v2.py</code>	BB_V2 signal/indicator engine; constructs the shared BBTradeManager with <code>strategy_id = BB_V2</code> .
<code>bb_trade_manager.py</code>	Shared with BB_V1 — same fill-resolution fix applies.

7. SCALP_V1 — Option-Selling Scalper

In plain words

SCALP_V1 sells options (a SHORT position) on 1-minute candles, one trade at a time. Selling is riskier to mismanage than buying: if your sell order has not filled yet but your safety net fires, you could accidentally open a position in the wrong direction. So the order of operations matters more here than anywhere else.

The flow: place the sell at a protected limit price, record that price immediately so the app knows it is in a trade, and place the safety-net GTT straight away using the stop-loss and target the signal already calculated. Then, in the background, confirm the real fill and update the recorded entry price. If the order is still unfilled after 50 seconds, cancel it — a scalp signal is only good for about a minute.

7.1 The worked example

Signal: sell 23000PE. Market price 100. The strategy has already calculated stop-loss = 110 and target = 85 (including any Max-SL cap — see 7.4). The app places a sell limit at $100 \times 0.99 = 99$, records entry ≈ 99 , and places a GTT with TP 85 / SL 110. The background thread later updates the recorded entry to the true fill (say 98.85). The GTT levels never move.

7.2 The live entry sequence

1. **Pre-checks** (trade_on, daily max-loss, session window, slot free).
2. **Place SELL limit** at $ltp \times 0.99$; the broker returns an order id and the limit price.
3. **Record provisionally**. Entry price = the limit price, mark in-trade, insert the DB row, send the entry alert.
4. **Place the GTT immediately** using the signal's SL/TP directly (these are fill-independent), mark the position PROTECTED.
5. **Background-confirm** the fill (poll up to 50s).

7.3 What the background thread decides

Situation	Action	Audit-log tag
COMPLETE	Update the recorded entry_price to the true fill (only if still the active trade). GTT untouched.	[SHORT] [FILL_CONFIRMED]
REJECTED / CANCELLED / LAPSED	Cancel the GTT, close the DB row, release the slot, alert. (Cancelling the GTT is vital — a stray BUY-back GTT on a non-existent short could open an unintended long.)	[SHORT] [DEAD_ENTRY]
Unfilled at 50s	Cancel the order, then re-check status to catch a fill that raced the cancel (see below).	[SHORT] [ENTRY_TIME_OUT]

The 50-second cancel and its race guard

A scalp signal is only valid for about one candle. If the sell has not filled in 50 seconds the order is cancelled. But a fill can land in the instant between the last check and the cancel taking effect, so after cancelling the app re-reads the order status:

- **Filled at the wire** (COMPLETE, full quantity) → treat as a normal fill: keep the position, the GTT stays, record the true fill. The slot is NOT released.
- **Partial fill** (e.g. 30 of 60) → the app does NOT act automatically. It logs loudly, alerts, and leaves the slot locked for manual handling — because a partial short with no full safety net needs a human decision.
- **Clean cancel** (nothing filled) → close the DB row as ENTRY_TIMEOUT and release the slot for the next signal.

Why the GTT can be placed before the fill here

Because SCALP's stop-loss and target come from the **signal** (calculated from candle data), not from the fill price, the GTT levels are known before the order even fills. So placing the GTT immediately is safe and correct — it does not depend on the fill at all. The fill only updates the recorded entry price for P&L.

7.4 Max-SL is respected, not recomputed

The strategy engine computes the stop-loss as “entry + risk × reward-ratio,” then caps it at “entry + max_sl_points” if a cap is configured (you will see MAX_SL_APPLIED in the log). The trade manager now uses that already-capped value **directly** and does not recompute it from the fill. Recomputing would silently discard your Max-SL cap — so it was removed.

Files involved

File	Role in this flow
trade_state_manager.py	on_sell_signal (entry), the background _confirm_fill_worker, _on_entry_filled / _on_entry_dead / _cancel_unfilled_entry, and the direction-aware exit (_force_exit).
signal_router.py	Routes a SELL signal to the slot's on_sell_signal under the single-trade gate (_entry_lock / _trade_reserved).
strategy_engine.py	Computes entry/SL/TP and applies the Max-SL cap (MAX_SL_APPLIED) before the signal is routed.
zerodha_executor.py	place_sell_entry, place_buy_exit, place_gtt_oco(direction="SHORT"), cancel_order, and get_order_fill.
zerodha_tick_engine.py	Builds candles, evaluates the signal, and calls route_sell_signal for the selected strike.

8. SCALP_V2 — Three-Leg Scalper

In plain words

SCALP_V2 is SCALP_V1's logic, but instead of selling one strike it sells three at once: the signal strike, plus the strikes one step above and one step below. The three are treated as a group, and the moment any one of them hits its target or stop, all three are closed together (“all-or-nothing”).

Each of the three legs follows the same fill-confirmation flow as SCALP_V1: place sell, record provisionally, place its GTT after its own fill confirms, and confirm in the background. The difference is purely that there are three of them, placed quickly one after another, without ever blocking the price feed.

8.1 How a group is built

1. **A SELL signal passes the gates** (trade_on, max-loss, session, dedup, selection filter) and claims the single “group” slot — one group at a time.
2. **Leg 1 (L1)** is the signal strike, using the signal's exact SL/TP (already Max-SL capped).
3. **Legs 2 and 3 (L2/L3)** are the +1 and −1 adjacent strikes. Their SL/TP are derived as the same percentage distance applied to each sibling's own premium.
4. **Each leg** places its sell, records the limit price, places its own GTT after its fill confirms, and runs its own background confirmation.

8.2 Per-leg fill confirmation (same shape as V1)

Situation	Action for that leg	Audit-log tag
COMPLETE	Place the leg's GTT (if not already), record the true fill.	[V2] [FILL_CONFIRMED] / [V2] [LEG_PROTECTED]
Dead	No GTT was placed; close the leg's DB row, drop the leg from the group, alert.	[V2] [DEAD_LEG]
Unfilled at 50s	Cancel; on a wire-fill protect it, on partial leave for manual, on clean cancel drop the leg.	[V2] [LEG_TIMEOUT]

All-or-nothing and a dropped leg

If a leg's order turns out dead or is cleanly cancelled, it is simply dropped from the group and the remaining real legs continue — they are genuine protected positions, and force-closing them because a different leg failed to enter would realise their cost for no reason. If a leg's price hits its TP/SL, all open legs close together.

8.3 Why the price feed never freezes

Leg placement is fast (place order, place GTT after its quick confirm) and each leg's **fill confirmation runs in its own background thread**. The group is marked OPEN as soon as legs are

placed, so the all-or-nothing tick monitor starts watching immediately while fills confirm in the background. The 1-minute scalp's tick processing is never blocked.

Files involved

File	Role in this flow
<code>scalp_v2_group_manager.py</code>	The whole group lifecycle: <code>try_enter</code> , <code>_fan_out</code> , <code>_place_leg</code> , the per-leg background workers (<code>_leg_fill_worker</code> , <code>_on_leg_filled</code> , <code>_on_leg_dead</code> , <code>_cancel_unfilled_leg</code>), all-or-nothing exit, and persistence.
<code>scalp_v2_tick_engine.py</code>	Builds candles, evaluates the signal, calls <code>group_manager.try_enter</code> , and forwards every tick to <code>group_manager.on_tick</code> for all-or-nothing exit.
<code>zerodha_executor.py</code>	Same order primitives as SCALP_V1, including <code>get_order_fill</code> .

9. HA_V1 — Heikin-Ashi (Self-Managed Exits)

In plain words

HA_V1 buys NIFTY weekly options based on Heikin-Ashi candle patterns. It is unusual in two ways. First, it tracks a large universe of strikes (~40) so that whichever strike gets selected already has warm candle history. Second, it does NOT use a broker GTT for protection — it watches the price itself and exits: take-profit is checked on every single tick, while stop-loss is checked only when a candle closes.

Because HA reacts to every tick, its entry must never make the app pause. The fix records the trade and starts watching it the instant the order is placed, and confirms the true fill quietly in the background. If the order turns out to be rejected, a background step removes the phantom and tells the engine to stop watching it.

9.1 Why HA must not block

HA's entry runs on the **tick thread** (inside candle-close handling). The previous code waited up to 120 seconds there for the fill — which would freeze take-profit monitoring for the entire ~40-strike universe while one entry waited. The fix moves fill confirmation to a background thread so the tick thread is released immediately.

9.2 The live entry sequence

1. **Pre-checks** (OFF gate, risk, valid risk distance).
2. **Compute TP** from the signal entry price (fill-independent); SL is the signal's red-candle low.
3. **Place the BUY limit** at $ltp \times 1.03$ (a 3% buffer so fast-moving option opens still fill).
4. **Record immediately** with the limit price as provisional entry, insert the DB row, and return success — the tick thread is free.
5. **Monitoring starts now:** the symbol is added to the active-trade set so TP (tick) and SL (candle close) are watched.
6. **Background-confirm** the fill (up to 120s).

9.3 What the background thread decides

Situation	Action	Audit-log tag
COMPLETE	Patch the recorded entry_price to the true fill (memory + DB).	[HA][LIVE] [FILL_CONFIRMED]
Dead	Remove the in-memory trade, close the DB row, call the engine's on_entry_dead() to stop monitoring and clear the signal flag, alert.	[HA] [DEAD_ENTRY] / [HA] [ENTRY_DEAD_ROLLBACK]
Timeout (120s)	Leave the provisional entry; log RECONCILE_NEEDED for startup reconcile.	[HA] [FILL_TIMEOUT]

9.4 Exit design (no GTT)

- **Take-profit** is checked on every tick. The moment price reaches the target, a limit sell is placed. This is why blocking the tick thread is unacceptable.
- **Stop-loss** is checked only on candle close — intra-candle wicks are deliberately ignored; only a candle that closes at or below the stop triggers the exit.
- **Both** are monitored for the currently selected strikes AND any strike with an open trade, because selection rotates every ~2 minutes and an open trade may sit on an older strike.

Trade-off to be aware of

Because HA has no broker GTT, a position has no broker-side protection if the application is down. This is pre-existing design, not introduced by the fix, but it is worth knowing: BB and SCALP positions are protected by a resting GTT even if the app stops; HA positions are not.

Files involved

File	Role in this flow
ha_trade_manager.py	Entry (_enter_live, now non-blocking), the background _confirm_fill_worker, _apply_confirmed_fill / _handle_dead_entry, and the tick/close exit checks and _exit_live.
ha_tick_engine.py	Universe discovery, candle building, signal evaluation, the active-trade set, and the new on_entry_dead() rollback hook.
ha_signal_engine.py	Heikin-Ashi 3-candle condition evaluation and per-side trade gating.
zerodha_executor.py	Order primitives and get_order_fill (HA degrades gracefully to get_last_avg_price if absent).
ha_candles_repo.py	Universe-wide HA candle storage and warmup.

10. Edge-Case Matrix (All Strategies)

This is the consolidated reference for “what happens if...”. Each row is a situation; the cells show how each strategy family handles it.

10.1 Order outcomes

Situation	BB_V1 / BB_V2	SCALP_V1 / V2	HA_V1
Order fills normally	Wait for COMPLETE, record true fill, then place GTT + DB row.	Record limit provisionally; GTT already placed from signal SL/TP; background updates entry to true fill.	Record limit provisionally, start monitoring; background updates entry to true fill.
Order rejected / cancelled / lapsed	Abort: no GTT, no DB row, warning alert. No phantom.	Background cancels the GTT, closes the DB row, releases the slot/leg, alert.	Background removes phantom, closes DB row, stops monitoring via on_entry_dead, alert.
Order not filled in time	120s timeout → best-effort price, GTT placed anyway, flagged for reconcile.	50s → cancel order (with race re-check). Clean cancel releases the slot/leg.	120s → leave provisional entry, flag RECONCILE_NEEDED.
Fill races the cancel/timeout	N/A (it blocks until COMPLETE).	Re-check after cancel: a wire-fill is kept & protected; the slot is NOT released.	N/A (no cancel; just records whatever the broker confirms).
Partial fill at cancel	N/A.	NOT auto-handled: log + alert, slot/leg left for manual intervention.	N/A.

10.2 Protection & price safety

Situation	Behaviour
Could a safety net ever fire on a non-existent position?	No. BB places the GTT only after a confirmed (or timed-out, possibly-real) fill. SCALP places the GTT from fill-independent signal levels but tears it down immediately on a dead order. HA has no GTT at all.
Is the recorded entry price ever wrong permanently?	Only if the order never confirms within the cap AND startup reconcile never runs. In normal operation the true fill replaces the provisional within seconds.
Stale cached price used for an exit?	Avoided: exits fetch a fresh REST LTP first and fall back to the cache only as a last resort.
Max-SL cap bypassed by recomputation?	No. SCALP uses the already-capped signal SL directly; the old recompute-from-fill step (which discarded the cap) was removed.
App restarts mid-trade	Startup reconciliation compares recorded trades against actual broker positions and closes/corrects mismatches. BB & SCALP positions also remain protected by their resting GTT while the app is down; HA positions are not (no GTT).

10.3 Concurrency guards

- **Single-trade gates:** SCALP uses an entry lock + reserved flag; BB checks in-trade state per side; SCALP_V2 allows one group at a time; HA gates per side. These prevent two orders racing into the same slot.
- **Stale-trade guards:** every background fill-confirmation step re-checks that the trade/leg it is about to modify is still the active one, so a trade that already exited cannot be corrupted by a late confirmation.
- **Double-exit guard (HA):** a per-side “exit in progress” flag prevents two rapid ticks from both firing a take-profit exit.

11. Side-by-Side Strategy Comparison

Aspect	BB_V1 / V2	SCALP_V1	SCALP_V2	HA_V1
Direction	LONG (buy)	SHORT (sell)	SHORT (sell)	LONG (buy)
Underlying	BANKNIFTY monthly	NIFTY/BANKNIFTY	NIFTY weekly	NIFTY weekly
Orders per signal	1 (split into 2 legs)	1	3 legs	1
Protection	GTT OCO	Inverted GTT OCO	Inverted GTT per leg	Self-managed (no GTT)
Fill model	Block until COMPLETE	Fill-first, GTT immediate	Per-leg, GTT after fill	Background, never blocks
Unfilled handling	120s timeout	50s cancel	50s cancel per leg	120s, keep provisional
TP check	GTT	GTT	GTT + tick monitor	Every tick
SL check	GTT	GTT	GTT + tick monitor	Candle close

12. Complete File Index

Every file touched or referenced by the live order flow, with its responsibility. Paths are under the backend application root.

12.1 Shared infrastructure

File	Responsibility
execution/zerodha_executor.py	All broker interaction: place orders (buy/sell entry, buy/sell exit, market sell), place/cancel GTT (LONG and inverted SHORT), read positions/orders, and get_order_fill (status + avg + filled qty). Routes writes through the relay; reads go direct.
execution/base_executor.py	Abstract executor interface that strategies code against.
db/trades_repo.py	Live-trade DB: insert, GTT link, close, direction-aware realized-P&L, fetch-by-id.
db/paper_trades_repo.py	Paper-trade DB equivalents and open-trade queries.
marketdata/ltp_store.py	In-memory latest-tick cache (with timestamp for staleness checks).
event_bus/audit_logger.py	The audit log every step writes to.

12.2 Per-strategy files

Strategy	Files
BB_V1	engine/bb_options/bb_trade_manager.py, bb_trade_state_manager.py, bb_tick_engine.py, option_selector.py, confluence_signal_engine.py
BB_V2	engine/bb_v2/bb_tick_engine_v2.py (+ shares bb_trade_manager.py with BB_V1)
SCALP_V1	trading/trade_state_manager.py, engine/signal_router.py, engine/strategy_engine.py, engine/zerodha_tick_engine.py
SCALP_V2	engine/scalp_v2/scalp_v2_group_manager.py, scalp_v2_tick_engine.py, scalp_v2_gtt_monitor.py, scalp_v2_live_eod.py
HA_V1	engine/ha_options/ha_trade_manager.py, ha_tick_engine.py, ha_signal_engine.py, db/ha_candles_repo.py, indicators/heikin_ashi.py, indicators/ema.py

12.3 Files changed by the fix

File	Change
zerodha_executor.py	Added get_order_fill (status + avg_price + filled_qty + pending_qty).
bb_trade_manager.py	Replaced the old 5-second fill poll with status-based _resolve_fill (block until COMPLETE/DEAD, 120s cap, dead-order abort).
trade_state_manager.py	Reworked on_sell_signal to Option A: provisional entry, immediate GTT from signal SL/TP, background fill confirm, 50s cancel with race + partial guards. Removed the fill-based SL recompute (preserves Max-SL).
scalp_v2_group_manager.py	Reworked _place_leg + per-leg background confirmation to the same Option A shape; GTT placed after each leg's fill.
ha_trade_manager.py	Non-blocking entry; background fill confirm; TP from signal entry; dead-order rollback via engine hook.

File	Change
ha_tick_engine.py	Added on_entry_dead rollback hook and engine back-reference wiring.

13. Audit-Log Cheat Sheet

When watching a live session, these are the tags that tell you an entry went well — or needs attention.

13.1 Healthy entry

Tag	Meaning
[BB][LIVE][ENTRY_CONFIRMED]	BB entry filled, GTT(s) placed, recorded.
[BB][LIVE][FILL_CONFIRMED]	BB saw the real fill price.
[SHORT][ENTRY_PROVISIONAL]	SCALP_V1 placed sell + GTT, awaiting fill confirmation.
[SHORT][FILL_CONFIRMED]	SCALP_V1 updated entry to the true fill.
[V2][LEG][LIVE] / [V2][LEG_PROTECTED]	SCALP_V2 leg placed and protected.
[HA][LIVE][ENTRY_PROVISIONAL] / [HA][LIVE][FILL_CONFIRMED]	HA placed and then confirmed the fill.

13.2 Needs attention

Tag	Meaning & action
[BB][LIVE][ENTRY_ORDER_DEAD]	BB buy rejected — no trade taken. Usually fine; check why (margin? trading disabled?).
[BB][LIVE][ENTRY_FILL_TIMEOUT]	BB couldn't confirm in 120s; position may be live & is protected with a best-effort price. Verify and let reconcile correct it.
[SHORT][DEAD_ENTRY] / [V2][DEAD_LEG]	Sell rejected; GTT cancelled, slot/leg released. Check the reason.
[SHORT][PARTIAL_FILL][MANUAL] / [V2][PARTIAL_FILL][MANUAL]	ACTION REQUIRED: a partial short with no full safety net. Handle manually; the slot/leg is intentionally left locked.
[SHORT][ENTRY_TIMEOUT] / [V2][LEG_TIMEOUT]	Sell unfilled at 50s; order cancelled. Slot released if nothing filled.
[HA][DEAD_ENTRY] / [HA][ENTRY_DEAD_ROLLBACK]	HA buy rejected; phantom removed and monitoring rolled back.
[*][RECONCILE_NEEDED]	Recorded entry is still the provisional estimate; startup reconcile will correct it.
[BB][LIVE][CRITICAL] GTT FAILED / [HA][GTT_FAIL]	Protection could not be placed — position relies on programmatic / EOD exit. Investigate promptly.

14. Pre-Live Testing Checklist

The failure-handling branches only execute under failure conditions, so they must be exercised deliberately before trusting them live. Recommended tests in a paper/test session:

1. **Normal fill:** confirm the recorded entry updates from the provisional/limit price to the true fill, and the GTT (where applicable) is placed.
2. **Rejected entry:** force a rejection (e.g. trading disabled mid-flight, or insufficient margin) and confirm the dead-order path fires — no GTT left behind, DB row closed, slot released, alert sent.
3. **Unfilled entry (SCALP):** place where the limit won't fill and confirm the 50s cancel, the post-cancel re-check, and slot release.
4. **Partial fill (SCALP):** if reproducible, confirm it logs `PARTIAL_FILL` [MANUAL, alerts, and leaves the slot locked (no auto-action)].
5. **HA non-blocking:** confirm that during a slow HA fill, take-profit on other strikes still fires (the tick thread is not blocked).
6. **Restart mid-trade:** restart the app with an open position and confirm reconciliation re-attaches or correctly closes it.

Deployment note

All strategies depend on `get_order_fill` in the executor. Deploy the updated `zerodha_executor.py` together with the strategy files. HA degrades gracefully without it, but SCALP and BB rely on it to distinguish a dead order from a pending one.

15. Quick Glossary

Term	One-line meaning
Limit price	The price you ask for; a request, not a guarantee.
Fill / average price	The price you actually got; the truth.
GTT	A resting safety-net order at the broker that fires on a price trigger.
OCO	One-Cancels-Other: a stop and a target together; one firing cancels the other.
Provisional entry	The limit price recorded immediately, later replaced by the real fill.
Dead order	An order the broker rejected, cancelled, or lapsed — no position resulted.
Phantom trade	A recorded trade for a position that does not actually exist; the fix prevents these.
Reconcile	Startup/periodic comparison of app records vs. broker reality, correcting mismatches.
Tick thread	The thread delivering price updates; must never be blocked by slow work.
Max-SL	A configured cap on how far the stop-loss may sit from entry.
All-or-nothing	SCALP_V2: when one leg exits, all legs in the group exit together.

— End of document —